

2. This is one way of computing the matrix / vector products. There are other ways too. You should check your answers by trying your solution in a Python IDE.

NOTE: These commands help you understand numpy data types and consequently their usage.
Print(`np.shape(x)`), print(`type(x)`) where `x = A, b`

- (a) `print(np.dot(A,b1)), print(np.dot(A,b2))`
- (b) `print(A*b1), print(A*b2.reshape(2,1))` (NOTE: `print(A*b2)` will give an error, Why?)
- (c) `print(A*b1+b2.reshape(2,1))`

3.

```
import numpy as np
```

```
def skew_symmetric(a):  
    return np.array([[0, -a[2], a[1]], \  
                    [a[2], 0, -a[0]], \  
                    [-a[1], a[0], 0]])
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
print("Cross product using skew-symmetric matrix:", np.dot(skew_symmetric(a), b))
```

```
print("Cross product using np.cross():", np.cross(a, b))
```

Solution printed

Cross product using skew-symmetric matrix: [-3 6 -3]

Cross product using `np.cross()`: [-3 6 -3]